To take this forward and translate everything into a working project, choose your preferred next step:

Option 1: Initialize REA to Inspect the Desktop App Bundle

If you have access to a compiled binary or packaged build of Creatorize (or want to inspect similar Electron/Node desktop tools using **morluto/rea**), run the setup command to hook your local coding agents into REA's MCP server:

```
Bash
npx -y rea-agents@latest setup
```

This registers REA with Claude Code, Cursor, or Codex, allowing your agent to run local application architecture analysis or deconstruct ASAR bundles.

Option 2: Write the Complete Boilerplate Code

We can build out the actual file structure for your local long-form generator repository. Would you like the code for:

1. **package.json & Electron Main Process Bridge** (spawning local Claude/Codex sessions and piping local stdout/stderr).
2. **The Three.js Chapter Builder Script** (procedurally generating camera paths, retro lighting rigs, and hard-pixel canvas exports).
3. **The Whisper + SciPy Alignment Python Script** (for millisecond-accurate word-level subtitles and scene splitting).

Tell me which piece you want to write next!